

Blocco 10 – Gentle Introduction: Window function

Sintassi, esempi guidati e soluzioni

Uso del materiale

Questo handout accompagna la presentazione aggiuntiva del blocco 10. Serve per introdurre gradualmente la sintassi del tema “Window function” e per collegare teoria, esempi e pratica.

1. Problema di partenza

Vogliamo ranking, totali progressivi e confronti con la riga precedente mantenendo le righe originali.

Ponte con i blocchi precedenti. GROUP BY comprime righe; le window function aggiungono calcoli mantenendo la granularità di partenza.

2. Sintassi essenziale

```
funzione(...) OVER (  
  PARTITION BY gruppo  
  ORDER BY ordinamento  
) AS nuova_colonna
```

3. Come leggere la sintassi

- OVER trasforma una funzione in funzione finestra.
- PARTITION BY definisce gruppi indipendenti.
- ORDER BY ordina le righe dentro la finestra.
- row_number, rank, lag sono funzioni analitiche comuni.
- La query conserva una riga per riga di input.

4. Esempi guidati

Ranking ordini per cliente

Domanda. Ranking ordini per cliente.

Soluzione.

```
SELECT customer_id,  
       order_id,  
       order_date,  
       gross_revenue,  
       row_number() OVER (  
         PARTITION BY customer_id  
         ORDER BY order_date, order_id  
       ) AS order_seq  
FROM order_revenue  
ORDER BY customer_id, order_seq;
```

Top prodotti per categoria

Domanda. Top prodotti per categoria.

Soluzione.

```
SELECT *
FROM (
    SELECT p.category,
           p.product_name,
           sum(oi.quantity) AS units,
           rank() OVER (
               PARTITION BY p.category
               ORDER BY sum(oi.quantity) DESC
           ) AS category_rank
    FROM order_items AS oi
    JOIN products AS p ON p.product_id = oi.product_id
    GROUP BY p.category, p.product_name
) AS ranked
WHERE category_rank <= 3
ORDER BY category, category_rank;
```

Ricavo progressivo per canale

Domanda. Ricavo progressivo per canale.

Soluzione.

```
SELECT channel,
       order_date,
       order_id,
       gross_revenue,
       sum(gross_revenue) OVER (
           PARTITION BY channel
           ORDER BY order_date, order_id
       ) AS running_revenue
FROM order_revenue
WHERE status NOT IN ('cancelled', 'refunded')
ORDER BY channel, order_date, order_id;
```

5. Esercizio con rappresentazione tabellare

Tabella di partenza.

cliente	ordine	data	ricavo
1	1	2025-01-06	1418.25
1	15	2025-04-03	335.00
1	31	2025-06-12	1470.90

Richiesta. Data la tabella ordini, costruire la query che assegna a ogni ordine il numero progressivo dell'ordine per cliente.

Procedimento atteso.

1. dichiarare la granularità del risultato;
2. scegliere la tabella o il passaggio di partenza;
3. aggiungere filtri, join, calcoli o subquery necessari;

4. ordinare il risultato in modo stabile;
5. scrivere una query di controllo.

6. Errori frequenti

- usare window function quando serve invece un riepilogo con `GROUP BY`;
- dimenticare `ORDER BY` nei calcoli progressivi;
- non distinguere `row_number` e `rank`;
- non specificare partizioni quando il confronto è per cliente o paese;
- interpretare il totale progressivo senza guardare l'ordinamento.

Checklist

- La domanda informale è stata tradotta in specifica?
- La granularità del risultato è dichiarata?
- I filtri e i collegamenti sono motivati?
- I nomi delle colonne finali sono leggibili?
- Esiste almeno una query di controllo?